Reporting Health Monitor - Complete Walkthrough

Purpose of This Script

This script simulates a simple **data quality monitoring framework** that a Data Analyst, Analytics Engineer, or Data Engineer might build.

The goal is to automatically detect common data problems before they appear in dashboards, reports, or machine learning models.

Examples of problems we want to catch:

- Missing values
- Duplicate primary keys
- Stale data
- Invalid categorical values
- Revenue mismatches between systems
- Unexpected row count drops

High-Level Flow

The script follows five major steps:

```
1. Create sample data
        ↓

2. Introduce intentional data issues
        ↓

3. Run data quality checks
        ↓

4. Collect test results
        ↓

5. Print a monitoring report
```

Step 1: Import Libraries

```
from __future__ import annotations
from dataclasses import dataclass
from typing import Iterable

import numpy as np
import pandas as pd
```

Why?

pandas

Used for:

- DataFrames
- Joins
- Filtering
- Aggregations
- Date handling

numpy

Used for:

- Random number generation
- Simulating realistic datasets

dataclass

Used to create simple objects that store data.

Instead of writing:

```
class CheckResult:
    def __init__(self, ...):
        ...
```

Python generates that automatically.

Step 2: Create the CheckResult Object

```
@dataclass
class CheckResult:
```

Every data quality test returns a CheckResult.

Example:

```
CheckResult(
    check_name="duplicate_primary_key",
    table_name="opportunities",
    status="fail",
    failed_rows=2,
    detail="opportunity_id has 2 duplicated rows."
)
```

This standardizes all monitoring output.

Step 3: Generate Sample Data

Function:

```
build_sample_data()
```

Creates four DataFrames:

```
accounts
opportunities
revenue_source
row_count_history
```

Accounts Table

Represents a typical dimension table.

Example:

account_id	account_name	segment
acct_0001	Sample Account 1	SMB
acct_0002	Sample Account 2	Enterprise

Account IDs

```
account_ids = [
    f"acct_{i:04d}"
    for i in range(1, 81)
]
```

Creates:

```
acct_0001
acct_0002
acct_0003
...
acct_0080
```

Opportunities Table

Represents a fact table.

Example:

opportunity_id	account_id	amount_reporting
opp_00001	acct_0001	50000
opp_00002	acct_0005	75000

Stage Names

Randomly assigned from:

```
[
    "Prospecting",
    "Discovery",
```

```
    "Evaluation",
    "Negotiation",
    "Closed Won",
    "Closed Lost",
    "Bad Stage"
]
```

Notice:

```
Bad Stage
```

is intentionally invalid.

This allows our monitoring framework to detect it later.

Step 4: Inject Data Quality Issues

The script intentionally creates bad data.

Without bad data every test would pass.

Duplicate Primary Key

```
opportunities.loc[5, "opportunity_id"] = (
    opportunities.loc[4, "opportunity_id"]
)
```

Before:

Row	opportunity_id
4	opp_00005
5	opp_00006

After:

Row	opportunity_id
4	opp_00005

Row	opportunity_id
5	opp_00005

Now we have a duplicate.

Missing Account ID

```
opportunities.loc[12, "account_id"] = None
```

Creates a NULL value.

Example:

opportunity_id	account_id
opp_00013	NULL

Missing Revenue

```
opportunities.loc[25, "amount_reporting"] = np.nan
```

Creates a missing numeric value.

Step 5: Create Revenue Source Table

```
revenue_source = (
    opportunities[
        ["opportunity_id", "amount_reporting"]
    ]
)
```

Creates a second table.

Think:

```
Reporting Layer
vs
Source System
```

Both initially match perfectly.

Step 6: Create Revenue Mismatches

```
mismatch_rows = (
    revenue_source
    .sample(8, random_state=seed)
    .index
)
```

Randomly selects 8 rows.

Example:

```
[3, 12, 25, 44, 87, 91, 120, 177]
```

Then:

```
revenue_source.loc[
    mismatch_rows,
    "amount_source"
] += rng.integers(...)
```

Changes those amounts.

Example:

Before:

```
50000
```

After:

```
52300
```

Now:

```
amount_reporting
≠
amount_source
```

This creates reconciliation failures.

Step 7: Create Historical Row Counts

```
row_count_history
```

Produces:

Date	Row Count
Day 1	172
Day 2	174
Day 3	177
Day 4	179
Day 5	181
Day 6	180
Day 7	139

Notice:

```
180 → 139
```

Large drop.

This simulates a broken ETL process.

Step 8: Required Field Check

Function:

```
check_required_fields()
```

Checks:

1. Column exists
2. Column contains no NULLs

Example:

```
account_id
```

If account_id contains:

```
NULL
```

Test fails.

Step 9: Duplicate Key Check

Function:

```
check_duplicate_key()
```

Uses:

```
df.duplicated()
```

Example:

account_id
1
2

account_id
2
3

Produces:

```
False
True
True
False
```

Count:

```
2 duplicates
```

Step 10: Freshness Check

Function:

```
check_freshness()
```

Finds:

```
newest_record = max(updated_at)
```

Then calculates:

```
today - newest_record
```

Example:

```
Today = June 10

Newest Record = June 7
```

Age:

```
3 days old
```

If threshold is:

```
2 days
```

Test fails.

Step 11: Valid Value Check

Function:

```
check_valid_values()
```

Checks if values belong to an approved set.

Valid:

```
Prospecting
Discovery
Evaluation
Negotiation
Closed Won
Closed Lost
```

Invalid:

```
Bad Stage
```

Test fails.

Step 12: Revenue Reconciliation

Function:

```
check_revenue_reconciliation()
```

Performs:

```
merge()
```

Equivalent SQL:

```
SELECT *
FROM opportunities o
LEFT JOIN revenue_source r
ON o.opportunity_id = r.opportunity_id
```

Calculates:

```
revenue_difference =
amount_reporting
-
amount_source
```

Example:

```
50000 - 52300
=
-2300
```

Absolute value:

```
.abs()
```

Becomes:

```
2300
```

Since:

```
2300 > 1
```

Mismatch detected.

Step 13: Row Count Drop Check

Function:

```
check_row_count_drop()
```

Compares:

```
Yesterday = 180
Today = 139
```

Calculation:

```
(180 - 139) / 180
```

Result:

```
22.8%
```

Threshold:

```
20%
```

Since:

```
22.8 > 20
```

Test fails.

Step 14: Collect Results

All test results are stored in:

```
results
```

Example:

```
[
    CheckResult(...),
    CheckResult(...),
    CheckResult(...)
]
```

Converted into a DataFrame:

```
results_df = pd.DataFrame(
    [r.__dict__ for r in results]
)
```

Result:

check_name	status
required_field_not_null	pass
duplicate_primary_key	fail
freshness	pass

Step 15: Calculate Health Score

```
score =
passed_checks
/
total_checks
*
100
```

Example:

```
15 checks total

12 pass

3 fail
```

Calculation:

```
12 / 15 = 0.80
```

Health Score:

```
80%
```

Step 16: Print Report

Function:

```
print_monitor_report()
```

Displays:

```
REPORTING HEALTH MONITOR

Health Score : 80%

Passed Checks: 12
Failed Checks: 3
```

Shows failed tests:

```
[FAIL]
duplicate_primary_key
```

```
[FAIL]
revenue_reconciliation

[FAIL]
row_count_drop
```

Shows mismatch details:

```
opportunity_id
amount_reporting
amount_source
revenue_difference
```

Allowing investigation of specific issues.

What This Script Teaches

This small project demonstrates many real-world data engineering concepts:

Data Quality

- Null checks
- Duplicate checks
- Domain validation

Data Monitoring

- Freshness
- Row count anomalies

Reconciliation

- Comparing reporting tables to source systems

Pandas Skills

- DataFrames
- merge()
- loc[]
- isna()
- duplicated()

- date_range()
- Timedelta()

Python Skills

- Functions
- Dataclasses
- Type hints
- List comprehensions
- f-strings

Real-World Equivalent

This is essentially a miniature version of:

- dbt tests
- Great Expectations
- Monte Carlo
- Soda
- Datadog Data Monitoring
- Custom Airflow quality checks

The difference is that you've built it yourself using only Python and Pandas.